PF 101 — Object-Oriented Programming

Performance Task

# SYSTEM PROPOSAL DOCUMENT

## *EcoWatch*

**Group No.:** *Group 6*

**Section:** *PF101 - A*

**Date Submitted:** *09/29/2026*

**Instructor:** *Harvey Sean Ocmen*

**SDG Focus:** *SDG 13, 14, 15*

# 1. Group Members

> ***Instructions:*** *List all five (5) members. Two members are already pre-selected — keep their names and add the remaining three. Mark exactly one member as Leader; this affects the grading formula below.*

| Full Name | Role | Section / Block | Task Assignment |
|---|---|---|---|
| Relampagos, Miguel Anton Roel T. | Leader | *BSIT - 3* | *Project Manager, UI and Database Lead Programmer* |
| Acero, Juniel | Member | *BSIT - 3* | *Documentation, Testing* |
| *Trumpeta, Ralph Adrianne A.* | *Member* | *BSIT - 3* | *UI Programmer, Testing* |
| *Deligero, Marchus* | *Member* | *BSIT - 3* | *Testing, Database Programmer* |

# 2. Project Name and Description

> ***Instructions:*** *State the working title of your system and a concise description (150–300 words) covering: the problem it solves, its target users, and how it applies Object-Oriented Programming.*

### Project Name

*EcoWatch - Environmental Monitoring and Reporting System*

### Project Description

*EcoWatch is a Java-based environmental monitoring and reporting system designed to help users record, organize, and track environmental concerns within their community. Environmental issues such as flooding, extreme weather events, coastal pollution, wildlife concerns, deforestation, and habitat damage can be difficult to organize when reports are recorded through disconnected methods. EcoWatch provides a centralized system where users can submit environmental reports, categorize them as climate, marine, or land-related concerns, provide relevant details, and monitor the status of each report.*

*The target users are community members and environmental observers who submit environmental reports, as well as authorized personnel who review reports and manage their progress. The system applies Object-Oriented Programming by representing users, environmental reports, locations, and follow-up actions as interacting objects. Inheritance and polymorphism can be applied to different types of environmental reports, while encapsulation and abstraction organize their data and behaviors. The system will use .txt files as a makeshift database through Java File I/O to store and manage records. CRUD operations will allow appropriate records to be created, viewed, updated, searched, and deleted.*

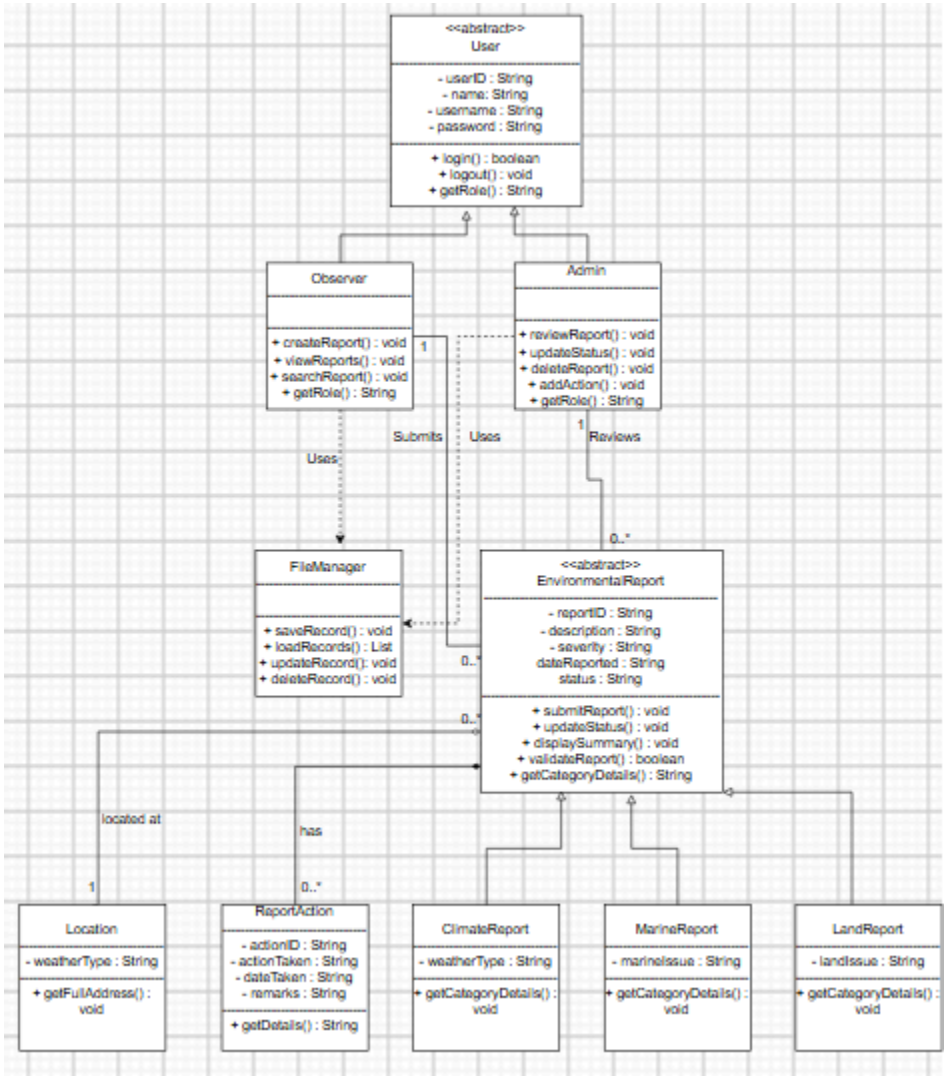
## 3. SDG and Number

***Instructions:*** *Identify the specific United Nations Sustainable Development Goal (SDG) your system is anchored to, and justify the alignment.*

| Field | Details |
|---|---|
| **SDG Number** | *SDG 13, SDG 14, SDG 15* |
| **SDG Title** | *Climate Action, Life Below Water, Life on Land* |
| **Alignment / Justification** | *EcoWatch is a reporting and monitoring system that supports the Sustainable Development Goals of climate action, life below water, and life on land by providing a platform that helps users monitor climate change concerns, marine problems, and land-related issues. The system lets users report environmental issues, organize collected data, and track reported problems over time, making monitoring easier and supporting better decision-making. Additionally, it organizes environmental data and makes it easier to track issues over time, helping improve environmental awareness, monitoring, and decision-making.* |

## 4. Class Diagram

***Instructions:*** *Insert your UML class diagram (classes, attributes, methods, and relationships). Then summarize how the four pillars of OOP are demonstrated in your design.*




## OOP Concepts Applied

| OOP Concept | Where Applied in the System |
|---|---|
| **Encapsulation** | *All attributes in User, EnvironmentalReport, Location, and ReportAction are private and accessed only through public getters, setters, and methods. For example, password in User and status in EnvironmentalReport cannot be changed directly; they are modified through login() and updateStatus(), which can validate the* |

| OOP Concept | Where Applied in the System |
|---|---|
| | *change first. validateReport() also protects data integrity before a report is saved to reports.txt.* |
| **Inheritance** | *Observer and Admin inherit userID, name, username, password, login(), and logout() from User. ClimateReport, MarineReport, and LandReport inherit reportID, description, severity, dateReported, status, and shared methods such as submitReport() and displaySummary() from EnvironmentalReport, and each adds only its own category attribute (weatherType, marineIssue, landIssue).* |
| **Polymorphism** | *getCategoryDetails() is declared once in EnvironmentalReport and overridden in ClimateReport, MarineReport, and LandReport, so a single call on an EnvironmentalReport reference returns the correct category-specific details at runtime. Likewise, getRole() is overridden in Observer and Admin, so the system can treat every logged-in user as a User and still get role-specific behavior.* |
| **Abstraction** | *User and EnvironmentalReport are abstract classes: they define what every user or report must have and do, but cannot be instantiated themselves. Only concrete types (Observer, Admin, and the three report categories) are created. FileManager also hides the File I/O details, so other classes call saveRecord() or loadRecords() without knowing how the .txt files are read or written.* |

## 5. Functionalities

***Instructions:*** *List the core features of your system. Each functionality should be backed by a .txt file acting as a makeshift database (read/write via Java file I/O) and should map to at least one OOP concept from Section 4.*

| # | Functionality | Description / Operations | Data File (.txt) |
|---|---|---|---|
| 1 | *User Registration & Login* | *Create (register account), Read (validate login), Update (change password). Users are Observer or Admin objects that inherit from User.* | *users.txt* |
| 2 | *Submit Environmental Report* | *Create. An Observer submits a Climate, Marine, or Land report with description, severity, date, and location; validateReport() checks the data before it is saved.* | *reports.txt, locations.txt* |
| 3 | *View & Search Reports* | *Read / Search. Users list reports and search by report ID, category, severity, or status; displaySummary() and getCategoryDetails() show category-specific details.* | *reports.txt* |

| # | Functionality | Description / Operations | Data File (.txt) |
|---|---|---|---|
| 4 | *Review, Update Status & Delete Report* | *Update / Delete. An Admin reviews a report, changes its status (e.g., Pending, Under Review, Resolved), or deletes invalid or duplicate reports.* | *reports.txt* |
| 5 | *Follow-up Actions* | *Create / Read / Update. An Admin records actions taken on a report (actionTaken, dateTaken, remarks), and users can view the action history of each report.* | *actions.txt* |

## . Grading Basis (For Instructor Use)

> ***Instructions:*** *This section is completed by the instructor. Individual member grades are computed separately using the accompanying Grading Workbook (Excel).*

| Component | Weight | Score |
|---|---|---|
| Proposal Documents | 50% | |
| Presentation, System Demonstration & Q&A | 50% | |
| Total | 100% | |